Welcome to Slidev

Presentation slides for developers

Press Space for next page →

What is Slidev?

Slidev is a slides maker and presenter designed for developers, consist of the following features

- 📝 **Text-based** - focus on the content with Markdown, and then style them later
- 🎨 **Themable** - theme can be shared and used with npm packages
- 🧑‍💻 **Developer Friendly** - code highlighting, live coding with autocompletion
- 🤹 **Interactive** - embedding Vue components to enhance your expressions
- 🎥 **Recording** - built-in recording and camera view
- 📤 **Portable** - export into PDF, PNGs, or even a hostable SPA
- 🛠️ **Hackable** - anything possible on a webpage

Read more about Why Slidev?

agenda

1. Welcome to Spring Batch
2. What is Slidev?
3. agenda
4. spring-batch
5. spring-batch-chunk
6. spring-batch-tasklet
7. spring-batch-reader
8. spring-batch-writer
9. spring-batch-processor
10. spring-batch-listener
11. spring-batch-partitioning
12. spring-batch-fault-tolerant
13. spring-batch-launch
14. spring-batch-metrics
15. spring-batch-configuring
16. spring-batch

spring-batch

provides reusable functions that are essential in processing large volumes of records, including
`logging/tracing`, `transaction management`, `job processing statistics`, `job restart`, `skip`, and `resource management`.
It also provides more advanced technical services and features that will enable extremely high-volume and high performance batch jobs through optimization and partitioning techniques.
Simple as well as complex, high-volume batch jobs can leverage the framework in a highly scalable manner to process significant volumes of information.

- Transaction management
- Chunk based processing
- Declarative I/O
- Start/Stop/Restart
- Retry/Skip
- Web based administration interface (Spring Cloud Data Flow)

spring-batch-chunk

ChunkOrientedTasklet

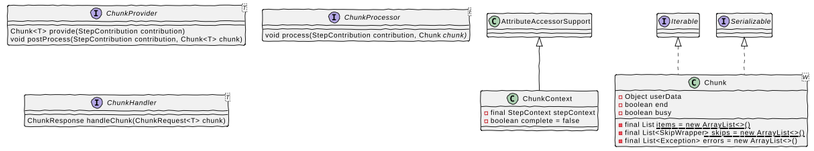
```
class ChunkOrientedTasklet<I> implements Tasklet {
  private final ChunkProcessor<I> chunkProcessor
  private final ChunkProvider<I> chunkProvider
  public ChunkOrientedTasklet(ChunkProvider<I> chunkProvider,ChunkProcessor<I> chunkProcessor){
    this.chunkProvider = chunkProvider
    this.chunkProcessor = chunkProcessor
  }
  public RepeatStatus execute(StepContribution contribution, ChunkContext chunkContext){
    Chunk<I> inputs = (Chunk<I>) chunkContext.getAttribute(INPUTS_KEY);
    chunkProcessor.process(contribution, inputs);
    chunkProvider.postProcess(contribution, inputs);
    chunkContext.removeAttribute(INPUTS_KEY);
    chunkContext.setComplete();
    return RepeatStatus.continueIf(!inputs.isEnd());
  }
}
```

ChunkProvider

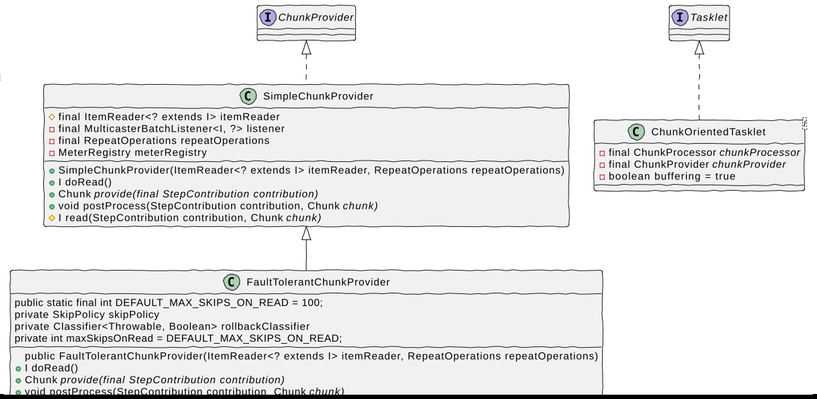
- SimpleChunkProvider
- FaultTolerantChunkProvider

```
public class FaultTolerantChunkProvider<I> extends SimpleChunkProvider<I> {
  public static final int DEFAULT_MAX_SKIPS_ON_READ = 100;
  private SkipPolicy skipPolicy = new LimitCheckingItemSkipPolicy();
  private Classifier<Throwable, Boolean> rollbackClassifier = new BinaryExceptionClassifier(true);
  private int maxSkipsOnRead = DEFAULT_MAX_SKIPS_ON_READ;
  public FaultTolerantChunkProvider(ItemReader<? extends I> itemReader, RepeatOperations repeatOperations) {
    super(itemReader, repeatOperations);
  }
}
```

Chunk, ChunkProvider, ChunkProcessor



ChunkProvider - SimpleChunkProvider, FaultTolerantChunkProvider



spring-batch-tasklet

TaskletStep

Chunk-oriented processing is not the only way to process in a Step. What if a Step must consist of a stored procedure call? You could implement the call as an `ItemReader` and return null after the procedure finishes. However, doing so is a bit unnatural, since there would need to be a no-op `ItemWriter`. Spring Batch provides the `TaskletStep` for this scenario.

```
class TaskletStep extends AbstractStep {
    - RepeatOperations stepOperations
    - CompositeChunkListener chunkListener
    - StepInterruptionPolicy interruptionPolicy
    - CompositeItemStream stream
    - PlatformTransactionManager transactionManager
    - TransactionAttribute transactionAttribute
    - Tasklet tasklet
    # void doExecute(StepExecution stepExecution)
}
```

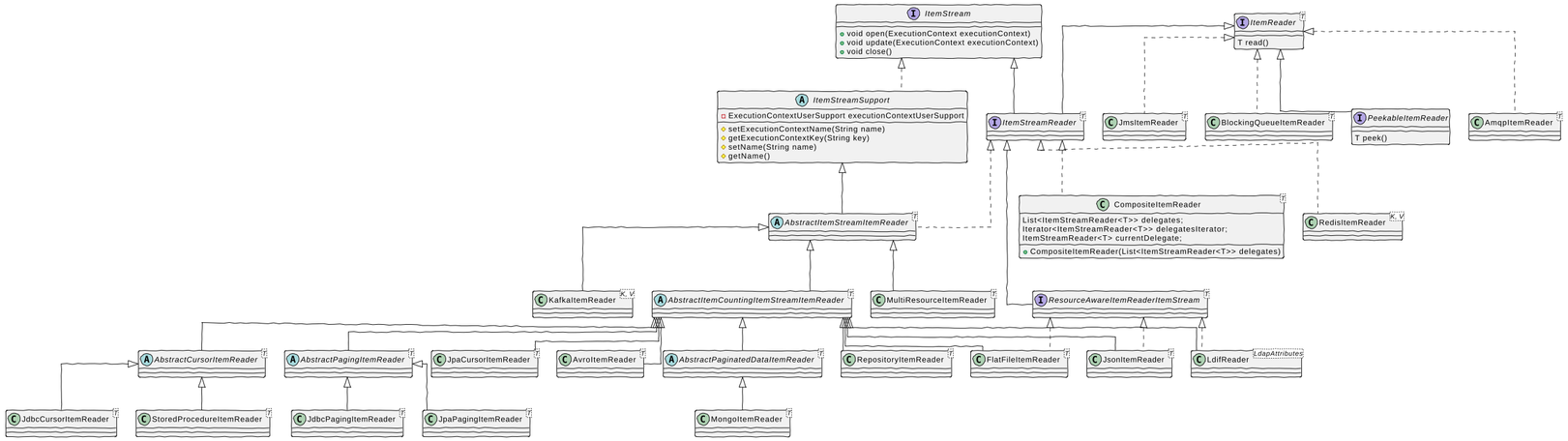
Tasklet

The `Tasklet` interface has one method, execute, which is called repeatedly by the `TaskletStep` until it either returns `RepeatStatus.FINISHED` or throws an exception to signal a failure. Each call to a Tasklet is wrapped in a transaction.

```
interface Tasklet {
    + RepeatStatus execute(StepContribution contribution, ChunkContext chunkContext)
}
```

spring-batch-reader

- SQL (jpa,jdbc,stored procedure), mq, amap, nosql
- File - json, csv, fixlength, xml
- Messaging - Kafka, Jms, Ldif, BlockingQueue

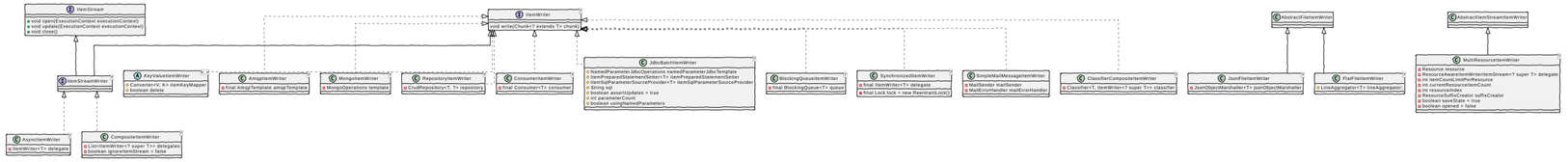


spring-batch-writer

- SQL (jpa,jdbc,stored procedure), mq, amap, nosql
- File - json, csv, fixlength, xml
- Messaging - Kafka, Jms, Ldif, BlockingQueue

```
interface ItemWriter<T> {
  void write(Chunk<? extends T> chunk)
}
```

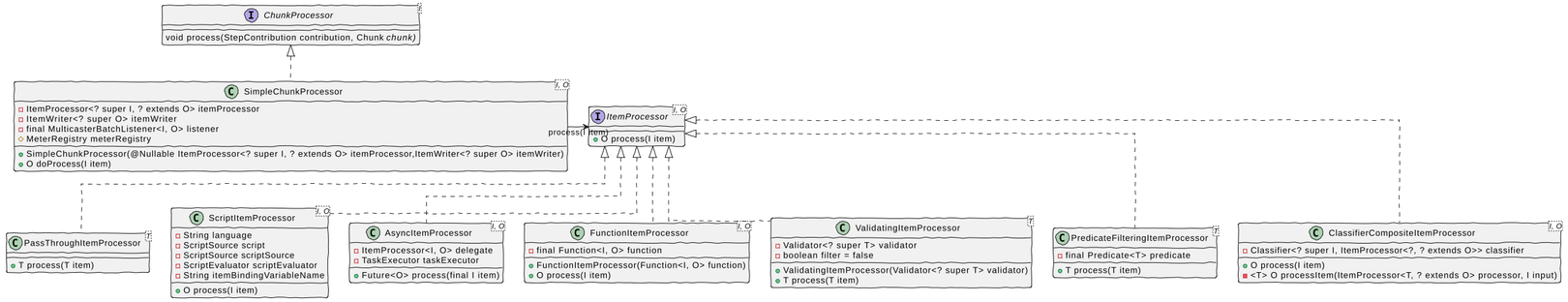
```
public interface ItemStream {
  default void open(ExecutionContext executionContext) throws ItemStreamException {}
  default void update(ExecutionContext executionContext) throws ItemStreamException {}
  default void close() throws ItemStreamException {}
}
```



spring-batch-processor

```java
public class SimpleChunkProcessor<I, O> implements ChunkProcessor<I>, InitializingBean {
  private ItemProcessor<? super I, ? extends O> itemProcessor;
  private ItemWriter<? super O> itemWriter;
  private final MulticasterBatchListener<I, O> listener = new MulticasterBatchListener<>();
  protected MeterRegistry meterRegistry = Metrics.globalRegistry;

  protected final O doProcess(I item) throws Exception {
    if (itemProcessor == null) {
      @SuppressWarnings("unchecked")
      O result = (O) item;
      return result;
    }
    try {
      listener.beforeProcess(item);
      O result = itemProcessor.process(item);
      listener.afterProcess(item, result);
      return result;
    }
    catch (Exception e) {
      listener.onProcessError(item, e);
      throw e;
    }
  }
}
```



spring-batch-listener

Listeners

- JobExecutionListener - @BeforeJob, @AfterJob
- StepExecutionListener - @BeforeStep, @AfterStep
- ChunkListener - @BeforeChunk, @AfterChunk, @AfterChunkError
- ItemReadListener, ItemProcessListener, ItemWriteListener
- SkipListener - @OnSkipInRead, @OnSkipInWrite, @OnSkipInProcess

JobExecutionListener

```
public interface JobExecutionListener {
  public void beforeJob(JobExecution jobExecution);
  public void afterJob(JobExecution jobExecution);
}
```

```
public class JobExecution extends Entity {
  private final JobParameters jobParameters;
  private JobInstance jobInstance;
  private volatile Collection<StepExecution> stepExecutions = Collections.synchronizedSet(new LinkedHashSet<>());
  private volatile BatchStatus status = BatchStatus.STARTING;
  private volatile LocalDateTime startTime;
  private volatile LocalDateTime createTime = LocalDateTime.now();
  private volatile LocalDateTime endTime;
  private volatile LocalDateTime lastUpdated;
  private volatile ExitStatus exitStatus = ExitStatus.UNKNOWN;
  private volatile ExecutionContext executionContext = new ExecutionContext();
  private transient volatile List<Throwable> failureExceptions
}
```

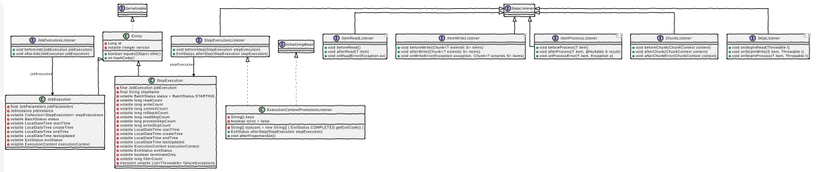
StepExecutionListener, ChunkListener

```
public interface StepExecutionListener extends StepListener {
  public void beforeStep(StepExecution stepExecution);
  public ExitStatus afterStep(StepExecution stepExecution);

}
```

```
public interface ChunkListener extends StepListener {
  public void beforeChunk(ChunkContext context);
  public void afterChunk(ChunkContext context);
  public void afterChunkError(ChunkContext context);
}
```

Item*Listener, SkipListener



spring-batch-partitioning

Scaling and Parallel Processing

- Multi-threaded Step (single-process)
- Parallel Steps (single-process)
- Remote Chunking of Step (multi-process)
- Partitioning a Step (single or multi-process)

Partitioning

```
public interface PartitionHandler {
  Collection<StepExecution> handle(StepExecutionSplitter stepSplitter, StepExecution stepExecution);
}
```

```
public class TaskExecutorPartitionHandler extends AbstractPartitionHandler
    implements StepHolder, InitializingBean {
    private TaskExecutor taskExecutor = new SyncTaskExecutor();
    private Step step;
    protected Set<StepExecution> doHandle(StepExecution managerExecution,Set<StepExecution> partitionExecutions) {}
    protected FutureTask<StepExecution> createTask(Step step, StepExecution stepExecution) {}
}
```

```
public class MultiResourcePartitioner implements Partitioner {
  private static final String DEFAULT_KEY_NAME = 'fileName';
  private static final String PARTITION_KEY = 'partition';
  private Resource[] resources = new Resource[0];
    public Map<String, ExecutionContext> partition(int gridSize) {}
}
```